

Table-by-Table Design

1.1 users

Represents people who use the platform.

Important fields

Field	Type	Purpose
<code>user_id</code>	UUID	Primary key
<code>name</code>	VARCHAR	User's name
<code>email</code>	VARCHAR	Login identifier
<code>password_hash</code>	VARCHAR	Encrypted/hashed password
<code>role</code>	VARCHAR	Doctor, Nurse, Records Staff, Admin
<code>created_at</code>	TIMESTAMP	Account creation

Why `role`?

Your PRD identifies different users.

Initially:

doctor
nurse
records_staff
admin

You don't need a separate `roles` table yet.

2. patients

This represents the **patient whose documents are being processed**.

This is separate from **users**.

That's important.

A doctor is a **user**.

A patient is a **subject of the documents**.

Fields

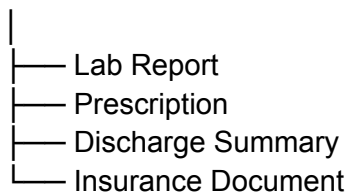
Field	Type	Purpose
patient_id	UUID	Primary key
patient_reference_id	VARCHAR	Internal patient identifier
name	VARCHAR	Patient name
date_of_birth	DATE	DOB
gender	VARCHAR	Gender
created_at	TIMESTAMP	Record creation

Why keep this table?

Without it, you'd have patient information duplicated across every document.

Instead:

Patient



This also makes future patient-level search much easier.

3. documents

This is the **central table**.

Almost everything revolves around documents.

Fields

Field	Type	Purpose
<code>document_id</code>	UUID	Primary key
<code>patient_id</code>	UUID	FK → patients
<code>uploaded_by</code>	UUID	FK → users
<code>file_name</code>	VARCHAR	Original filename
<code>file_type</code>	VARCHAR	PDF, PNG, JPG etc.
<code>file_url</code>	TEXT	Location of actual file
<code>document_type</code>	VARCHAR	Prescription, Lab Report etc.
<code>processing_status</code>	VARCHAR	Uploaded, Processing, Completed, Failed
<code>ocr_quality_score</code>	FLOAT	OCR quality assessment
<code>uploaded_at</code>	TIMESTAMP	Upload time
<code>processed_at</code>	TIMESTAMP	Completion time

Example

`document_id`: D123
`patient_id`: P101
`uploaded_by`: U20
`file_name`: blood_report.pdf
`file_type`: pdf
`document_type`: lab_report
`processing_status`: completed
`ocr_quality_score`: 0.94

4. extractions

This represents information extracted from a document.

This is where I'd deliberately avoid creating tables like:

medications
diagnoses
lab_tests
doctors
hospitals
...

for Version 1.

That would unnecessarily complicate the database.

Instead, use **JSONB**.

Fields

Field	Type	Purpose
<code>extraction_id</code>	UUID	Primary key
<code>document_id</code>	UUID	FK → documents
<code>extracted_data</code>	JSONB	Structured extracted information
<code>confidence_score</code>	FLOAT	Overall extraction confidence
<code>verification_status</code>	VARCHAR	Pending / Approved / Edited
<code>verified_by</code>	UUID	FK → users
<code>created_at</code>	TIMESTAMP	Extraction time
<code>verified_at</code>	TIMESTAMP	Verification time

Example `extracted_data`

```
{  
  "patient_name": "John Doe",  
}
```

```
"diagnosis": [
  "Type 2 Diabetes"
],
"medications": [
  {
    "name": "Metformin",
    "dosage": "500 mg",
    "frequency": "Twice daily"
  }
],
"lab_tests": [
  {
    "name": "HbA1c",
    "value": "7.2",
    "unit": "%"
  }
]
}
```

This fits your **schema-based extraction** feature while keeping the database flexible.

5. summaries

Stores AI-generated document summaries.

Fields

Field	Type	Purpose
summary_id	UUID	Primary key
document_id	UUID	FK → documents
summary_text	TEXT	Generated summary
model_name	VARCHAR	Model used

`created_at` `TIMESTAMP` Generation time

Example:

`document_id` → D123

summary:

"Patient's HbA1c level is 7.2%..."

Why separate it?

Because the summary is conceptually different from extracted structured data.

You may also regenerate the summary later using another model.

6. `document_chunks`

This is the important table for your **RAG system**.

Documents need to be divided into smaller chunks before embedding.

Document

↓

Chunk 1

Chunk 2

Chunk 3

Chunk 4

↓

Embeddings

Fields

Field	Type	Purpose
<code>chunk_id</code>	UUID	Primary key

<code>document_id</code>	UUID	FK → documents
<code>chunk_index</code>	INT	Chunk ordering
<code>content</code>	TEXT	Chunk text
<code>embedding</code>	VECTOR	Embedding
<code>page_number</code>	INT	Source page
<code>created_at</code>	TIMESTAMP	Creation time

Why `page_number`?

This directly supports your **source citation** feature.

The RAG system can return:

Source: blood_report.pdf, Page 3

rather than simply:

Source: blood_report.pdf

That's useful for your project.

7. conversations

Represents a user's Q&A session.

Fields

Field	Type	Purpose
<code>conversation_id</code>	UUID	Primary key

<code>user_id</code>	UUID	FK → users
<code>patient_id</code>	UUID	FK → patients
<code>created_at</code>	TIMESTAMP	Start time
<code>updated_at</code>	TIMESTAMP	Last activity

Example:

Doctor



Patient: P101



Conversation #45

8. messages

Stores individual chat messages.

Fields

Field	Type	Purpose
<code>message_id</code>	UUID	Primary key
<code>conversation_id</code>	UUID	FK → conversations
<code>role</code>	VARCHAR	user / assistant
<code>content</code>	TEXT	Message
<code>citations</code>	JSONB	Sources used
<code>confidence_score</code>	FLOAT	AI confidence
<code>created_at</code>	TIMESTAMP	Message time

Example assistant message


```
{
  "content": "The patient's HbA1c was 7.2%",
  "citations": [
    {
      "document_id": "D123",
      "page": 3
    }
  ],
  "confidence_score": 0.94
}
```

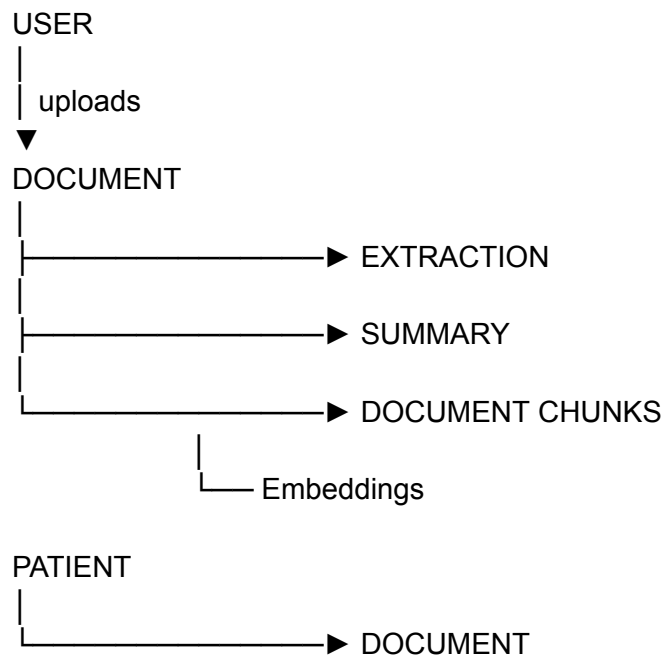
This supports:

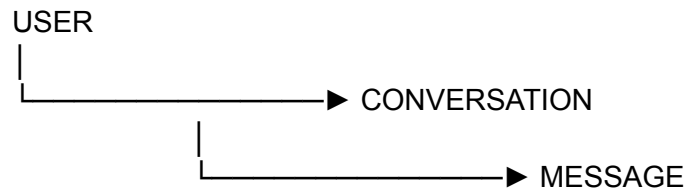
- RAG
- citations
- confidence scores
- chat history

without requiring additional tables.

9. Relationship Overview

The central relationship is:





This is deliberately simple.

10. How This Supports Your Core Features

This is probably the **most important part of your database design justification**.

Core Feature	Database Support
Document Upload	<code>documents</code>
OCR Processing	<code>documents.ocr_quality_score</code> + processed file
Document Classification	<code>documents.document_type</code>
Structured Extraction	<code>extractions.extracted_data</code>
Confidence-aware Extraction	<code>extractions.confidence_score</code>
Human Verification	<code>extractions.verification_status</code> , <code>verified_by</code>
Summarization	<code>summaries</code>
Intelligent Search	<code>document_chunks</code> + indexes
Semantic Search	<code>document_chunks.embedding</code>
Hybrid Search	PostgreSQL full-text search + <code>embedding</code>
RAG	<code>document_chunks</code>
Source Citations	<code>page_number</code> + <code>messages.citations</code>

AI Confidence	<code>messages.confidence_score</code>
Q&A	<code>conversations + messages</code>
Patient Information	<code>patients</code>
User Management	<code>users</code>

So **every database component has a reason to exist.**

11. What We Are Deliberately NOT Creating

This is just as important for keeping your database simple.

I would **not** create these tables right now:

- ✗ `diagnoses`
- ✗ `medications`
- ✗ `lab_tests`
- ✗ `hospitals`
- ✗ `doctors`
- ✗ `insurance_claims`
- ✗ `audit_logs`
- ✗ `knowledge_graph`
- ✗ `patient_timeline`
- ✗ `document_versions`
- ✗ `notification_preferences`
- ✗ `model_registry`

Why?

Because they're either:

- unnecessary for the current PRD,
- better represented inside JSONB,
- future functionality,
- or infrastructure concerns rather than core product data.

For example, instead of:

MEDICATIONS
DIAGNOSES
LAB_TESTS

we currently have:

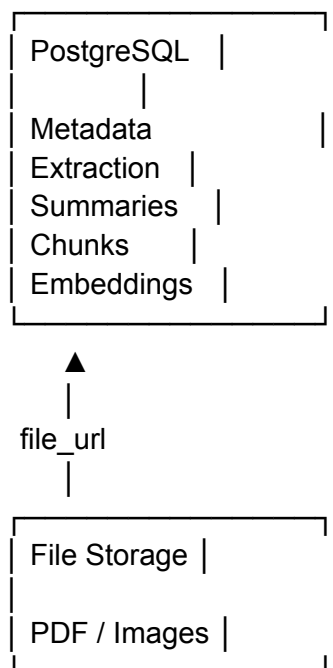
EXTRACTIONS
└─ extracted_data (JSONB)

If your project later needs sophisticated analytics on medications or lab values, you can normalize those into separate tables.

That's what makes the design **extensible without overengineering Version 1**.

12. Important Design Decision: Don't Store Files in the DB

The architecture should be:



This prevents the PostgreSQL database from becoming unnecessarily large.

13. Recommended Technology Choice

For your current project, I'd recommend:

Database

PostgreSQL

Vector Search

pgvector

File Storage

Local filesystem initially

Later:

AWS S3 / Azure Blob Storage

Why PostgreSQL + pgvector?

You could use:

PostgreSQL

+

ChromaDB

but that means your backend has to manage two databases.

For your specific requirement:

simple + scalable + easy backend integration

I'd start with:

FastAPI



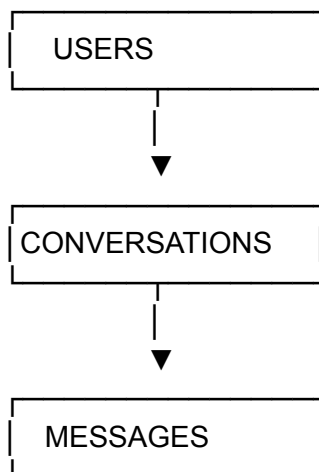
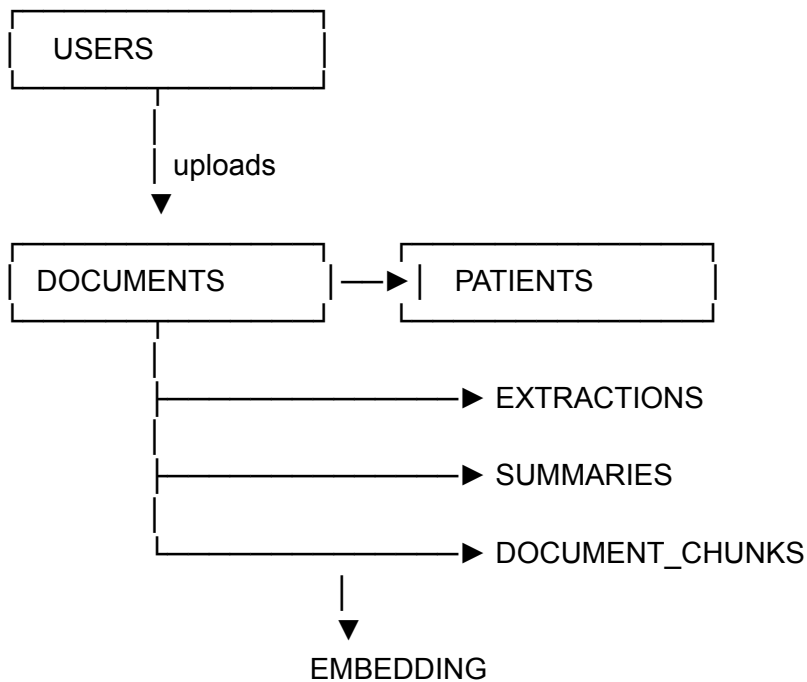
PostgreSQL

Relational Data
JSONB
Full-text Search
pgvector Embeddings

One primary database makes the system significantly easier to deploy and maintain.

14. Final Proposed Schema

So your initial database can be summarized as:



8 tables. That's it.

For the current IPS18 requirements, I would consider this a **good stopping point**.

It is:

- **Simple** → 8 core tables
- **Complete** → covers all current core features
- **Scalable** → UUIDs, indexing, JSONB, pgvector
- **Backend-friendly** → clear FK relationships and predictable entities
- **Extensible** → future features can be added without redesigning everything
- **Aligned with the PRD** → every major feature maps to a database component
- **Not overengineered** → no unnecessary micro-tables or knowledge graph

ER Diagram:

